

Using the IP Catalog and IP Integrator

Objectives

- After completing this lab, you will be able to:
- Include an IP in the project during the project creation
- Use IP Catalog to generate a clocking core
- Instantiate the generated clock
- Create a block design using IP Integrator
- Instantiate the block design
- Generate bitstream and verify the functionality in hardware

Design Description

The design used in this lab is a programmable waveform generator, also known as a signal generator.

The waveform generator in this design is intended to be a “standalone” device that is controlled via a PC (or other terminal device) using RS-232 serial communication. The design described here implements the RS-232 communication channel, the waveform generator and connection to the external DAC, and a simple parser to implement a small number of “commands” to control the waveform generation.

The wave generator implements a look-up table of 1024 samples of 16 bits each in a RAM. The wave generator also implements three variables:

- **nsamp**: The number of samples to use for the output waveform. Must be between 1 and 1024.
- **prescale**: The prescaler for the sample clock. Must be 32 or greater.
- **speed**: The speed (or rate) for the output samples in units of the prescaled clock.

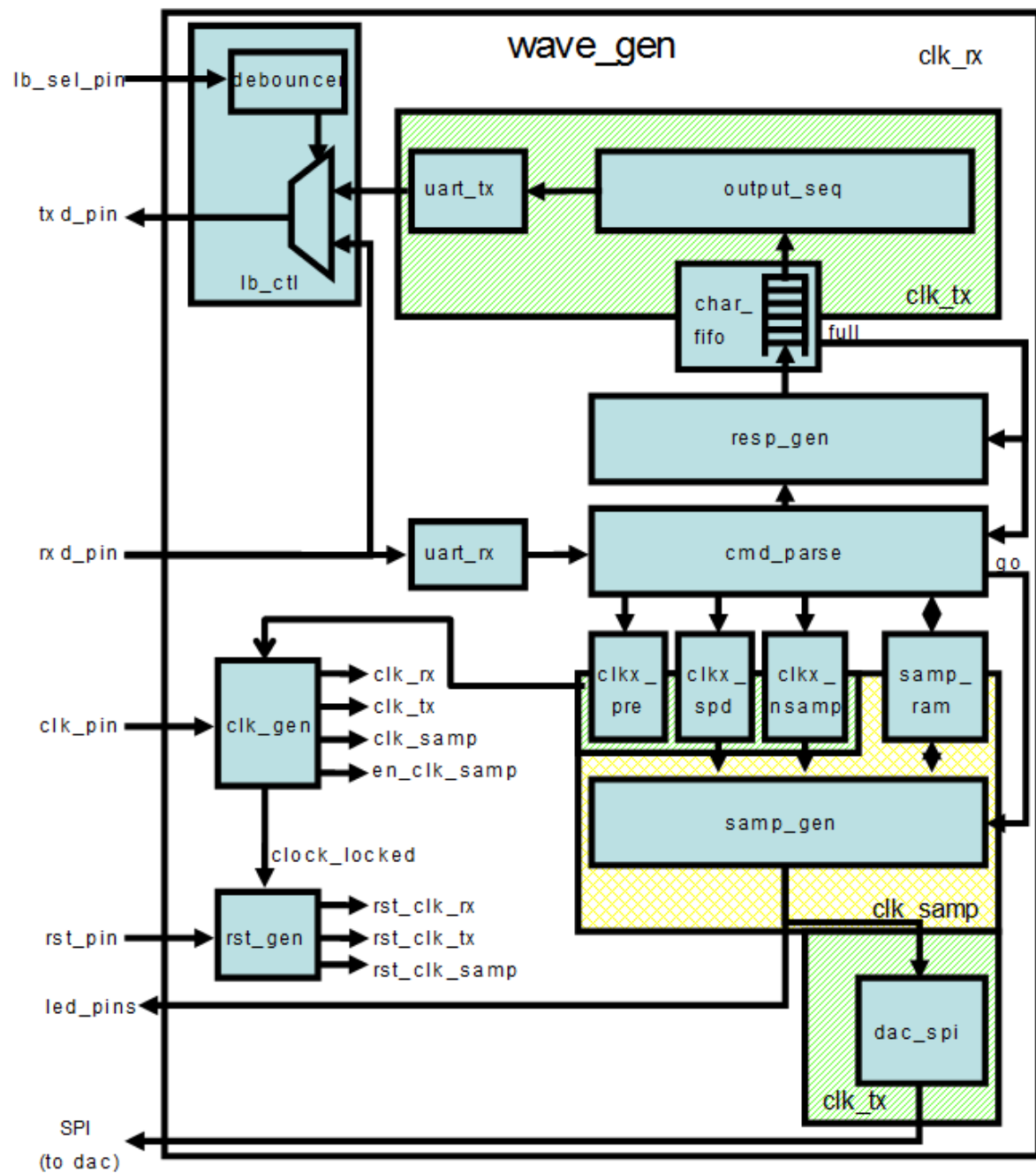
The wave generator can be instructed to send the appropriate number of samples once, cycling from 0 to nsamp-1 once and then stopping, or continuously, where it continuously loops the nsamp samples. When enabled, either once or continuously, the wave generator will send one sample to the DAC every (prescale x speed) clk_tx clock cycles. The contents of the RAM, as well as the three variables, can be changed via commands sent over the RS-232 link, as can the mode of the wave generator. The wave generator will generate responses for all commands.

There are three clock domains within this design: **clk_rx**, **clk_tx**, and **clk_samp**. The clock generator module instantiates all the clocking resources required for generating these three clocks. All three clocks are derived from a single clock input, coming in on **clk_pin**. The frequency of the clock input is 125MHz for PYNQ-Z1 and PYNQ-Z2.

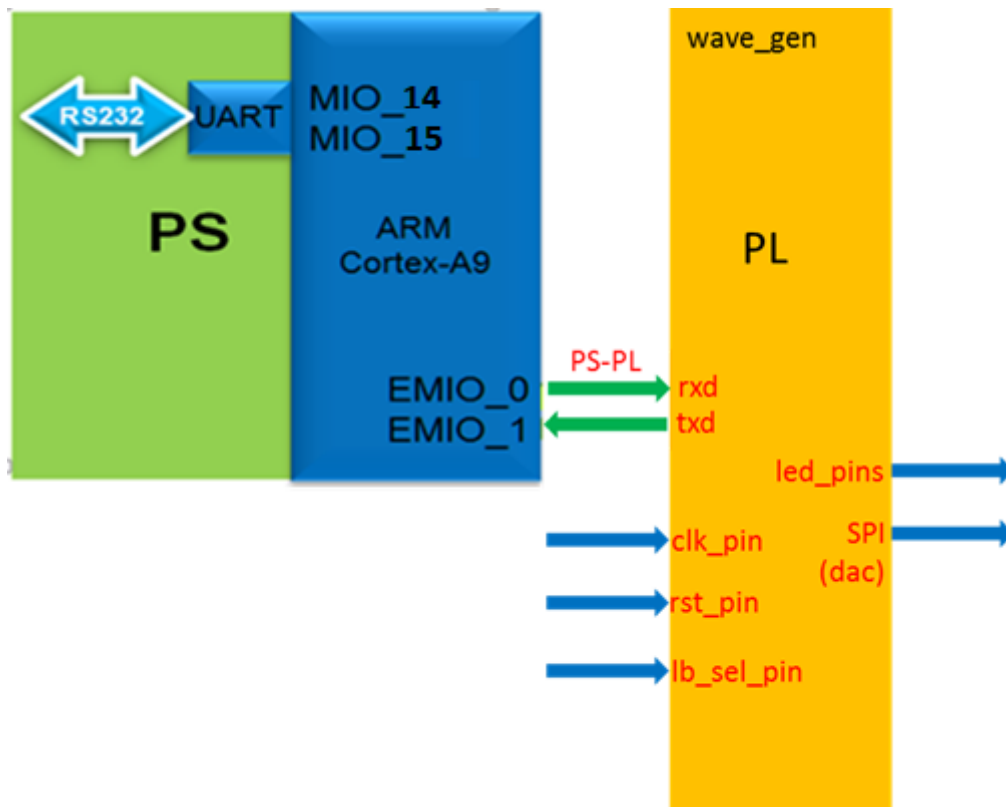
In this design we will use board's USB-UART which is controlled by the Zynq's ARM Cortex-A9 processor. Our PL design needs access to this USB-UART. So first thing we will do is to create a Processing System design which will put the USB-UART connections in a simple GPIO-style and make it available to the PL section.

The provided design places the UART (RX and TX) pins of the PS (Processing System) on the Cortex-A9 in a simple GPIO mode to allow the UART to be connected (passed through) to the Programmable Logic. The processor samples the RX signal and sends it to the EMIO channel 0 which is connected to Rx input of the HDL module provided in the Static directory. Similarly, the

design samples the Tx output of the HDL module through another EMIO channel 1 and sends it on the PS UART TX pin. This is done through a software application provided in the lab4.sdk folder hierarchy.



The complete design on PL



The Complete System

Steps

Create a Vivado Project using IDE

In this design we will use board's USB-UART which is controlled by the Zynq's ARM Cortex-A9 processor. Our PL design needs access to this USB-UART. So first thing we will do is to create a Processing System design which will put the USB-UART connections in a simple GPIO-style and make it available to the PL section.

Launch Vivado and create a project targeting the XC7Z020clg400-1 device, and use the provided tcl script file (ps7_create_pynq.tcl) to generate the block design for the PS subsystem. Also, add the Verilog HDL files, wave_gen_pins_pynq.xdc and wave_gen_timing_pynq.xdc files from the <2018_2_zynq_sources>\lab4 directory.

1. Open Vivado by selecting **Start > Xilinx Design Tools > Vivado 2018.2**
2. Click **Create New Project** to start the wizard. You will see *Create A New Vivado Project* dialog box. Click **Next**.
3. Click the Browse button of the *Project location* field of the **New Project** form, browse to **<2018_2_zynq_labs>**, and click **Select**.
4. Enter **lab4** in the *Project name* field. Make sure that the *Create Project Subdirectory* box is checked. Click **Next**.
5. Select **RTL Project** option in the *Project Type* form, and click **Next**.
6. Using the drop-down button, select **Verilog** as the *Target Language* and *Simulator Language* in the *Add Sources* form.

7. Click on the **Blue Plus** button, then the **Add Files...** button and browse to the **<2018_2_zynq_sources>\lab4** directory, select all the Verilog files, click **OK**, and then click **Next**.
8. Click **Next** to get to the *Add Constraints* form.
9. Click on the **Blue Plus** button, then **Add Files...** and browse to the **c:\xup\fpga_flow\2018_2_zynq_sources\lab4** directory (if necessary), select *wave_gen_timing_pynq.xdc* and the appropriate *wave_gen_pins_pynq.xdc* and click **Open**.
10. Click **Next**.
11. In the *Default Part* form, Use the **Boards** option, you may select the **PYNQ-Z1** or the **PYNQ-Z2** depending on your board from the Display Name drop down field.

You may also use the **Parts** option and various drop-down fields of the **Filter** section and select the **XC7Z020clg400-1** part.
12. Click **Next**.
13. Click **Finish** to create the Vivado project.

Correct the errors by adding file.

1. You will notice *Syntax Error Files* are being highlighted in the **Sources** pane.
If you check the messages Tab, you will see that these errors are due to missing files.
2. Click on **Add Sources** in the *Flow Navigator* pane.
3. Select *Add or Create Design Sources* and click **Next**.
4. Click on the **Blue Plus** button, then the **Add Files...** button and browse to **<2018_2_zynq_sources>\lab4\pynq\ip**
5. In the *File Type* field, select **All Files**, and then select **char_fifo.xcix** file.
6. Click on the **Blue Plus** button, then the **Add Files...** button and browse to **<2018_2_zynq_sources>\lab4**
7. In the *File Type* field, select **All Files**, and then select **clogb2.txt** file.
8. Click **OK** and then **Finish**.
9. In the *Sources* pane, expand *Design Sources* and *wave_gen_top* and *wave_gen* if necessary, and double-click on the **clk_gen_i0** entry.

Scroll down the file and notice that around line 79 there is an instruction to instantiate a clock core.
10. In the Tcl Shell window enter the following command to change to the lab directory and hit **Enter**.

`cd C:/xup/fpga_flow/2018_2_zynq_sources/lab4`
11. Generate the PS design by executing the provided Tcl script.

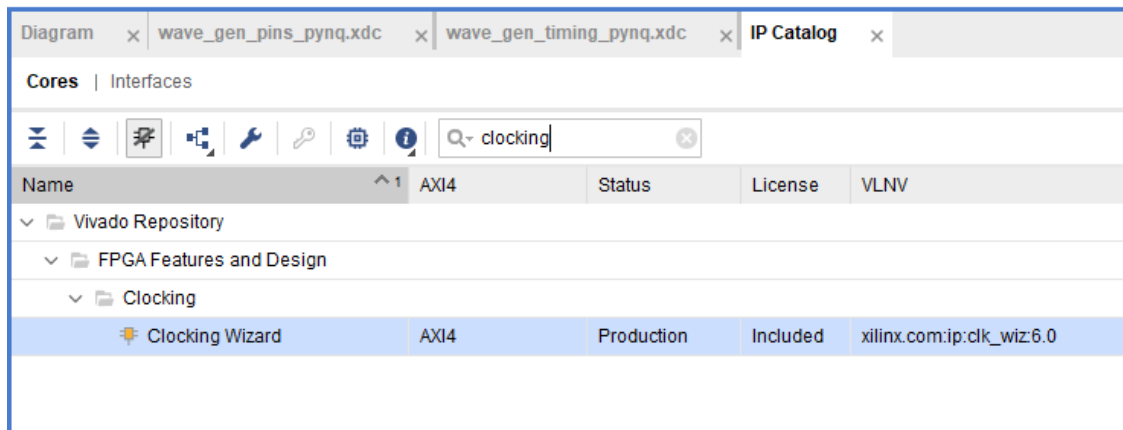
`source ps7_create_pynq.tcl`

This script will create a block design called *system*, instantiate ZYNQ PS, enable two GPIO channels (GPIO14 and GPIO15) and two EMIO channels. It will create *system.bd* that is instantiated under *system_wrapper.v* wrapper file. You can check the contents of the tcl files to confirm the commands that are being run.

Generate and Instantiate Clock Generator Module

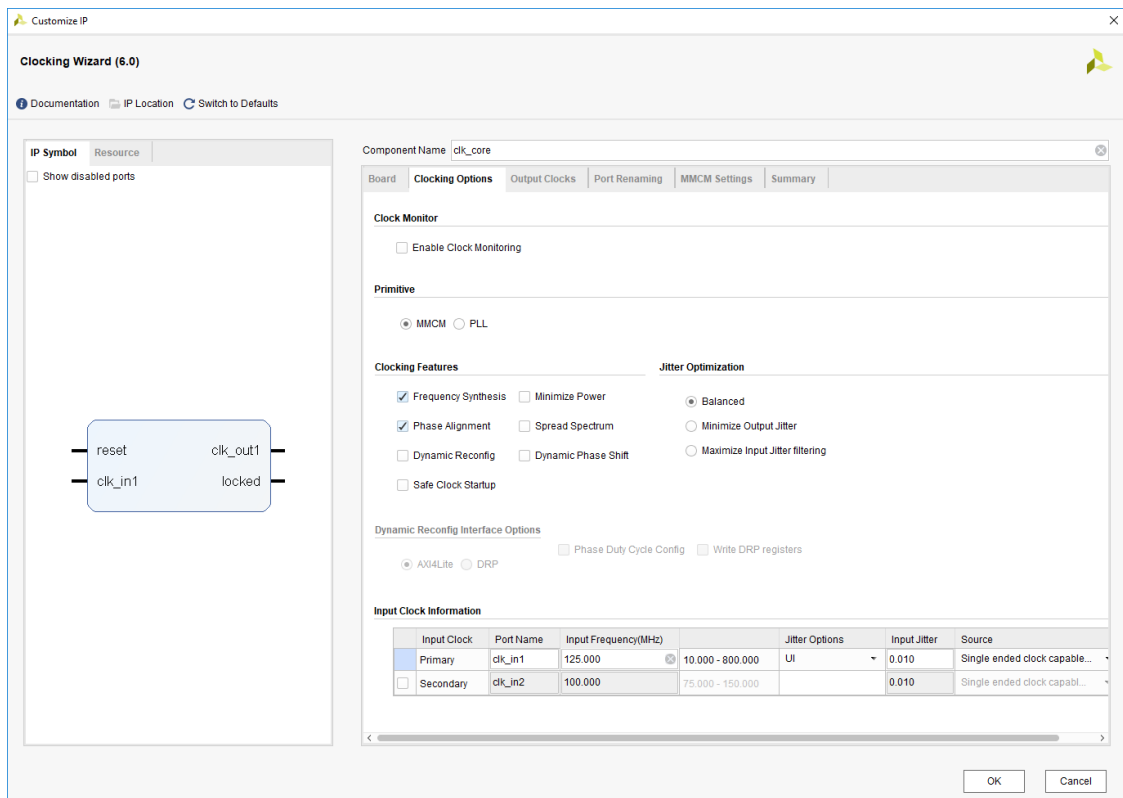
Launch the clocking wizard from the IP Catalog of Vivado and generate the clock core with input frequency of 125.00 MHz and two output clocks of 50.000 MHz each.

1. Click on **IP Catalog** in the *Flow Navigator* pane. The IP Catalog will open in the auxiliary pane.
2. Expand the **FPGA Features and Design > Clocking** sub-folders and double-click on the **Clocking Wizard** entry.



Accessing the clocking wizard

3. Click **Customize IP** on the following Add IP window. The clocking wizard will open.
4. Change the core name to **clk_core**. Make sure that the *Primary* input clock frequency is **125.000** MHz and the primitive used is **MMCM**.



The clocking wizard

5. Select the **Output Clocks** tab. Click on the check box to enable the second clock output. Make sure that the requested output frequency is 50 MHz for both clocks.

Board	Clocking Options	Output Clocks	Port Renaming	MMCM Settings	Summary			
The phase is calculated relative to the active input clock.								
Output Clock	Port Name	Output Freq (MHz)		Phase (degrees)		Duty Cycle (%)		Drives
		Requested	Actual	Requested	Actual	Requested	Actual	
☒ clk_out1	clk_out1	50.000	50.000	0.000	0.000	50.000	50.0	BUFG
☒ clk_out2	clk_out2	50.000	50.000	0.000	0.000	50.000	50.0	BUFG

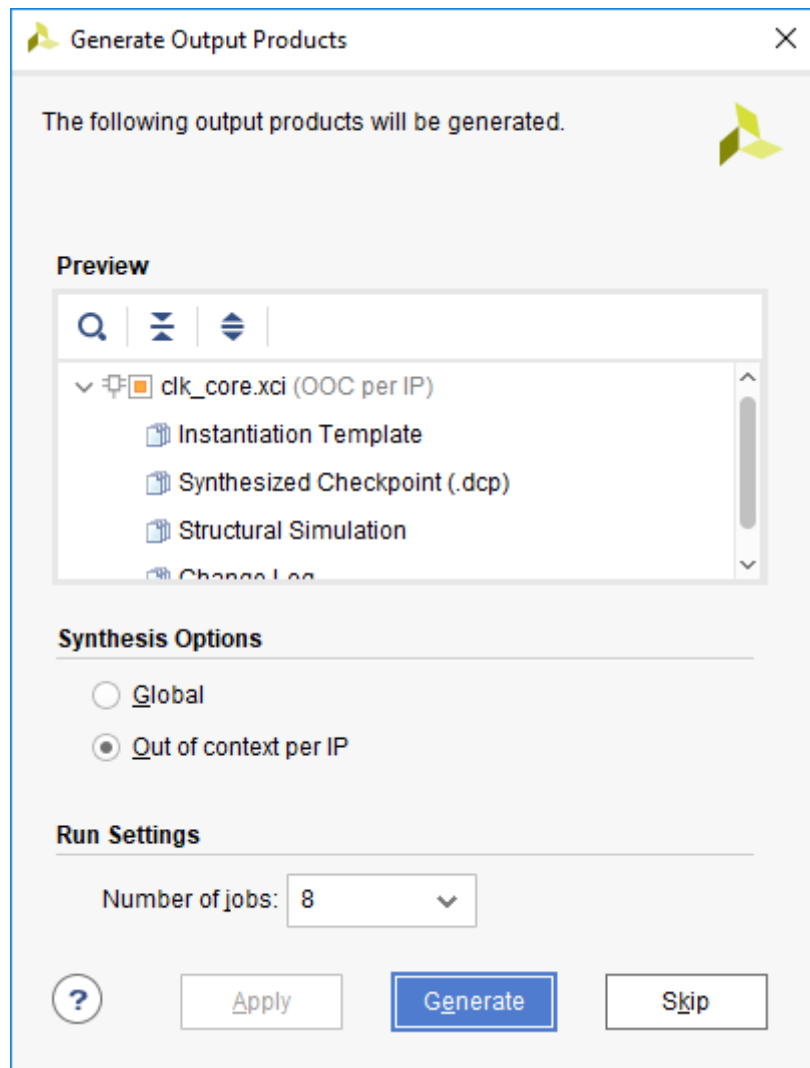
Setting output clocks

6. Click on the **Summary** tab and check the information.

Board	Clocking Options	Output Clocks	Port Renaming	MMCM Settings	Summary			
Primary Input Clock Attributes								
Input Clock Frequency (MHz)					125.000			
Clock Source					Single_ended_clock_capable_pin			
Jitter					0.010			
Clocking Primitive Attributes								
Primitive Instantiated : MMCM								
Divide Counter : 1								
Mult Counter : 8.000								
Clock Phase Shift : None								
Clock Wiz O/p Pins	Source	Divider Value	Tspread (ps)	Pk-to-Pk Jitter (ps)	Phase Error (ps)			
clk_out1	MMCM CLKOUT0	20.000	OFF	143.688	96.948			
clk_out2	MMCM CLKOUT1	20	OFF	143.688	96.948			
clk_out3	OFF	OFF	OFF	OFF	OFF			
clk_out4	OFF	OFF	OFF	OFF	OFF			
clk_out5	OFF	OFF	OFF	OFF	OFF			
clk_out6	OFF	OFF	OFF	OFF	OFF			
clk_out7	OFF	OFF	OFF	OFF	OFF			

Summary page of the clock core being generated

7. Click **OK** to see the *Generate Output Products* form.



Generate output products form

8. Click on **Generate** to generate the output products including the instantiation template. Click **OK** to proceed.

Instantiate the generated clock core.

1. Select the **IP Sources** tab in the *Sources* pane.
2. Expand the **IP (2)** branch. Notice the two IP entries. The *char_fifo* IP is the core that was included while creating project. The second core *clk_core* is the one that you have generated.
3. Expand **clk_core > Instantiation Template** and double-click on **clk_core.veo** to see the instantiation template.
4. Copy lines 72 through 81 and paste them at or around line 79 of the **clk_gen.v** file.
5. Change the instance name and net names to as shown in the figure below to match the names of existing signals in the design.

```

78 // Instantiate clk_core - generated by the Clocking Wizard
79 clk_core clk_core_i0
80 (
81 // Clock out ports
82 .clk_out1(clk_rx), // output clk_out1
83 .clk_out2(clk_tx), // output clk_out2
84 // Status and control signals
85 .reset(rst_i), // input reset
86 .locked(clock_locked), // output locked
87 // Clock in ports
88 .clk_in1(clk_pin)); // input clk_in1
89 // Insert Clock Core here
90

```

Assigning instance name and net connections

6. Click () to save **clk_gen.v**
7. Select the *Hierarchy* tab, expand the **wave_gen > clk_gen_i0** hierarchy and verify that **clk_core.xci** is in the hierarchy. The IP has a bordered yellow square icon next to it.

```

▼ ● wave_gen_top (wave_gen_top.v) (2)
  ▼ ● U0_wave_gen : wave_gen (wave_gen.v) (14)
    ▼ ● clk_gen_i0 : clk_gen (clk_gen.v) (2)
      ● clk_div_i0 : clk_div (clk_div.v)
      >  clk_core_i0 : clk_core (clk_core.xci)
      > ● rst_gen_i0 : rst_gen (rst_gen.v) (3)

```

The clk_core instantiated and shown in the hierarchy

Implement the Design

1. Click on the **Run Implementation** in the *Flow Navigator* pane.
2. Click **OK** and run the synthesis first before running the implementation process. Click **Save** to Save Project if prompted.

When the implementation is completed, a dialog box will appear with three options.

3. Select the *Open Implemented Design* option and click **YES**.

View the amount of FPGA resources consumed by the design using Report Utilization.

1. In the *Flow Navigator* pane, select **Open Implemented Design > Report Utilization**.
The Report Utilization dialog box opens.
2. Click OK.
3. Verify that the design is using the clock resource.

Tcl ConsoleMessagesLogReportsDesign RunsTimingPowerMethodologyDRCUtilization

Q≡≡

Q≡≡%

MMCM2_ADV

▼ Clocking

BUFHCE (1%)

BUFGCTRL (9%)

MMCM2_ADV (25%)

Specific Feature

Primitives

Name	Used
▼ N wave_gen_top	1
▼ U0_wave_gen (wave_gen)	1
▼ clk_gen_i0 (clk_gen)	1
> clk_core_i0 (clk_core)	1

Clock resource utilization

Generate the Bitstream and Verify the Functionality

1. In the Flow Navigator, under Program and Debug, click Generate Bitstream.
2. Click **Cancel** when the bitstream generation is completed.

Connect the board and power it ON. Open a hardware session, and program the FPGA.

1. Make sure that the Micro-USB cable is connected to the JTAG PROG.
2. Select the *Open Hardware Manager* option and click **OK**.
The Hardware Manager window will open indicating “unconnected” status.
3. Click on the **Open target** link, then **Auto Connect** from the dropdown menu.
You can also click on the **Open recent target** link if the board was already targeted before.
4. The Hardware Manager status changes from Unconnected to the server name and the device is highlighted. Also notice that the Status indicates that it is not programmed.
5. Select the device and verify that the **wave_gen_top.bit** is selected as the programming file in the General tab.

Start a terminal emulator program such as TeraTerm or HyperTerminal. Select an appropriate COM port (you can find the correct COM number using the Control Panel). Set the COM port for 115200 baud rate communication. Program the FPGA .

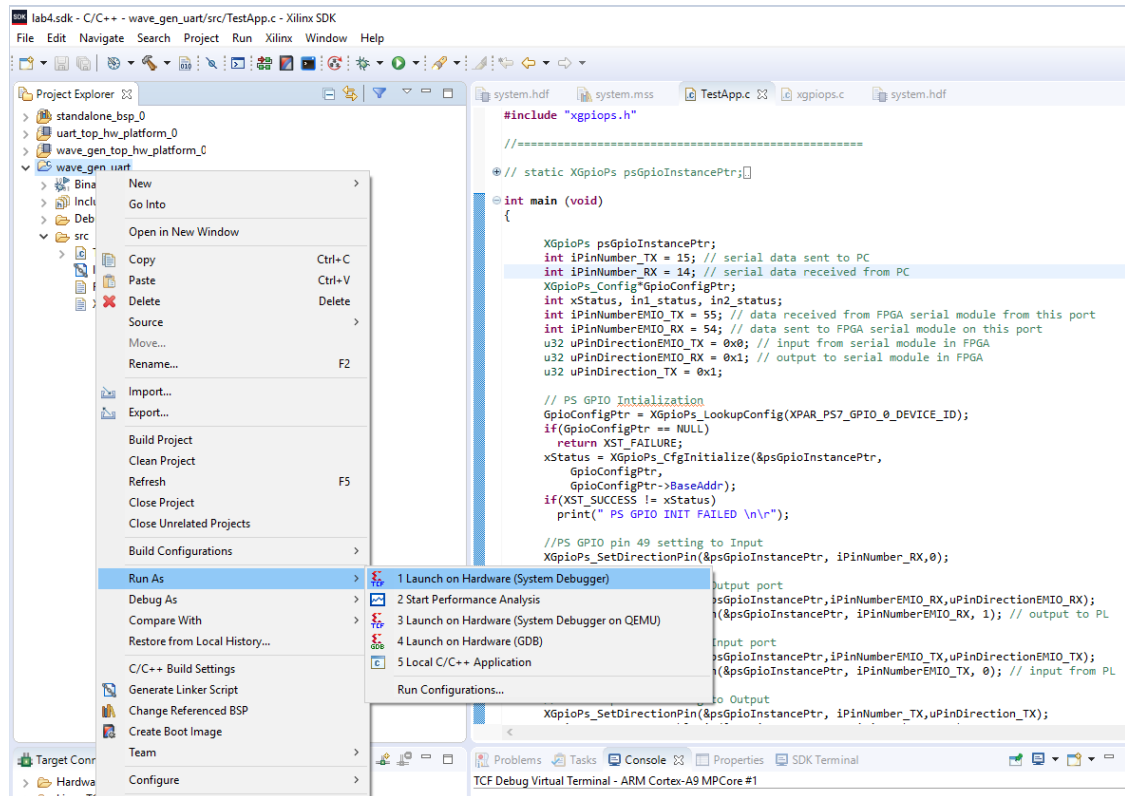
1. Start a terminal emulator program such as TeraTerm or HyperTerminal.
2. elect the appropriate COM port (you can find the correct COM number using the Control Panel).
3. Set the *COM* port for **115200** baud rate communication.
4. Right-click on the FPGA entry in the Hardware window and select **Program Device...**
5. Click on the **Program** button.

The programming bit file be downloaded and the DONE light will be turned ON indicating the FPGA has been programmed.

Start a SDK session, point it to the `c:/xup/fpga_flow/2018_2_zynq_sources/lab4/pynq/lab4.sdk` workspace.

1. Open **SDK** by selecting **Start > Xilinx Design Tools > Xilinx SDK 2018.2**
2. In the **Select a workspace** window, click on the browse button, browse to `C:\xup\fpga_flow\2018_2_zynq_sources\lab4\pynq\lab4.sdk` and click **OK**.
3. Click **OK**.

In the *Project Explorer*, right-click on the `wave_gen_uart`, select **Run As**, and then **Launch on Hardware (System Debugger)**



Running the application

4. Slide *Switch 0* to the **ON** position and type in some characters in the terminal window and see the character is echoed back. Setting *Switch 0* to the ON position makes the design function as a loopback.
5. Select **File > Send File ...** in the Tera Term window.
6. Browse to `<2018_2_zynq_sources>\lab4`, select **testpattern.txt** file, and click **Open**.

The file content will be send to the design. The file content is as follows:

```
*PFFFF          <-- specifies the pre-scaling
*S0fff          <-- specifies the speed value
*N000f          <-- specifies the number of samples to play
*W00000000      <-- write first sample of value 0 at location 0000
*W00011111      <-- write second sample of value 0x1111 at location 0001
```

*W00022222
 *W00033333
 *W00044444
 *W00055555
 *W00066666
 *W00077777
 *W00088888
 *W00099999
 *W000AAAAA
 *W000BBBBB
 *W000CCCCC
 *W000DDDDD
 *W000EEEEEE
 *W000FFFFF

7. The design understands various commands as listed in the figure below. All values are in hexadecimal. All values and addresses are in hexadecimal.

Cmd	Input	Response	Description
*W	aaaavvvv	-OK or -ERR	03ff ≥ aaaa ≥ 0000. Value "vvvv" is written into RAM at location "aaaa" and "-OK" is return.
*R	aaaa	-hhhh dddd or -ERR	03ff ≥ aaaa ≥ 0000. If in range, then the value at "aaaa" is returned in hex and decimal.
*N	vvvv	-OK or -ERR	0400 ≥ vvvv ≥ 0001. Specifies the number of samples before recycling.
*P	vvvv	-OK or -ERR	ffff ≥ vvvv ≥ 0020. Specifies prescaling value to divide <i>clk_tx</i> by to produce <i>clk_samp</i> .
*S	vvvv	-OK or -ERR	ffff ≥ vvvv ≥ 0001. Specifies "speed" value to divide <i>clk_samp</i> by to produce the rate of read from RAM.
*n/*p/*s		-hhhh dddd	Returns current value of nsamp, prescale, and speed.
*G		-OK	Triggers a single pass through nsamp memory locations.
*C		-OK	Starts continuous triggering.
*H		-OK	Halts continuous loop at end of current cycle.

Commands

- Type *G in the terminal window and observe the LED pattern changing slowly as written by the above file
- You can type *s to see the sample value, *p to see the prescale value, and *n to see how many samples are being played back.
- You can also type *H to halt the play.

```

*PFFFF-OK
*S0fff-OK
*N000f-OK
*40000000-OK
*40001111-OK
*40002222-OK
*40003333-OK
*40004444-OK
*40005555-OK
*40006666-OK
*40007777-OK
*40008888-OK
*40009999-OK
*4000AAAA-OK
*4000BBBB-OK
*4000CCCC-OK
*4000DDDD-OK
*4000EEEE-OK
*4000FFFF-OK
*G-OK
*H-OK

```

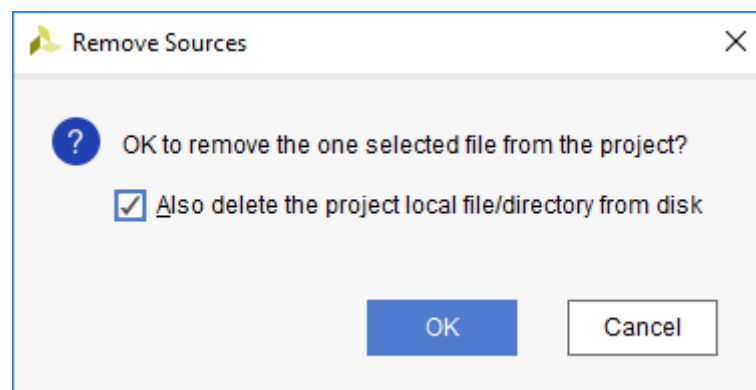
Terminal window display

11. Select **File > Close Hardware Manager**. Click **OK** to close it.
12. Close the **SDK** program by selecting **File > Exit** and click **OK**.

Generate and Instantiate an IPI Block

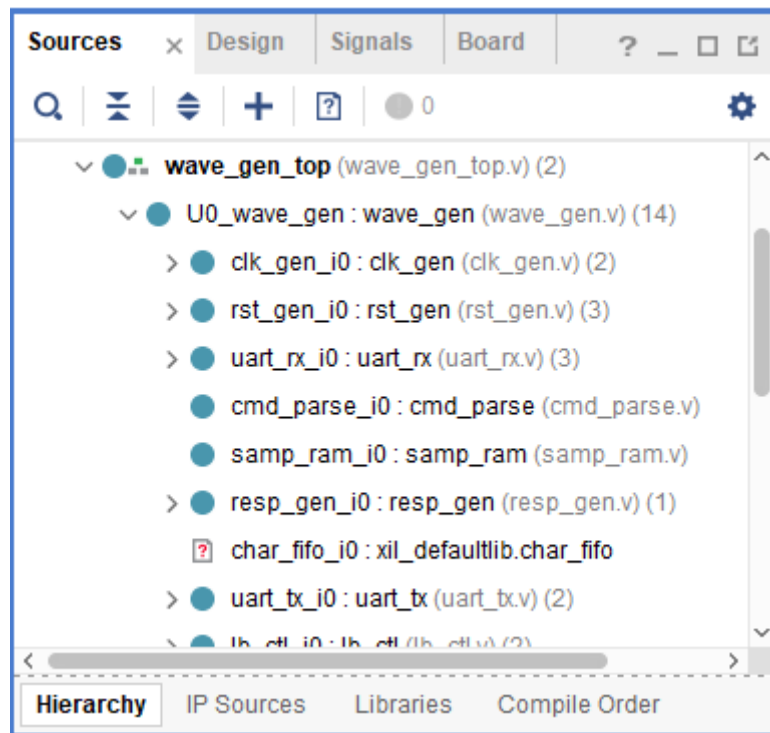
Save the project as lab4_ipi. Remove the char_fifo IP from the design.

1. Select **File > Save Project As...** and save it as **lab4_ipi** in the **<2018_2_zynq_labs>** directory making sure that the *Create Project Subdirectory* option is checked.
2. Select the **IP Sources** tab in the *Sources* pane.
3. Right-click on *char_fifo*, and select **Remove File from Project...**
4. If a following window appears, click on the check-box of *Also delete the project local file/directory from disk*, and click **OK** or else continue to next step.



Removing an existing IP from the project

5. Select **Hierarchy** tab in the *Sources* pane and observe that the *char_fifo* instance has a ? mark indicating that it is missing the source file.



Removed source file

6. Double-click on the **wave_gen.v** to open it in the editor window.
7. Toggle line comments the instantiation of the char_fifo from the file around line 336.

```

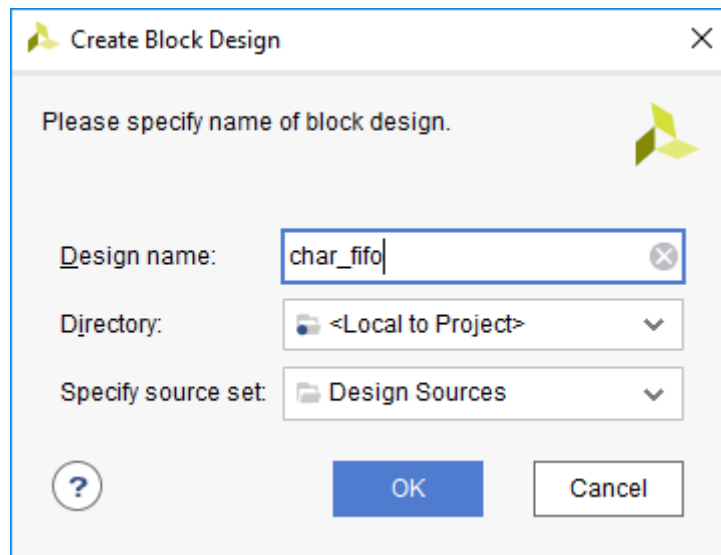
336 // char_fifo char_fifo_i0 (
337 //     .din      (char_fifo_din), // Bus [7 : 0]
338 //     .rd_clk   (clk_tx),
339 //     .rd_en    (char_fifo_rd_en),
340 //     .rst      (rst_i),          // ASYNCHRONOUS reset - to both sides
341 //     .wr_clk   (clk_rx),
342 //     .wr_en    (char_fifo_wr_en),
343 //     .dout     (char_fifo_dout), // Bus [7 : 0]
344 //     .empty    (char_fifo_empty),
345 //     .full     (char_fifo_full)
346 //);

```

8. Select **File > Save File**.

Create a block design naming it as char_fifo and add an instance of an FIFO Generator IP.

1. Click on **Create Block Design** in the Flow Navigator block.
2. Enter **char_fifo** as the block design name.



Naming the new block design

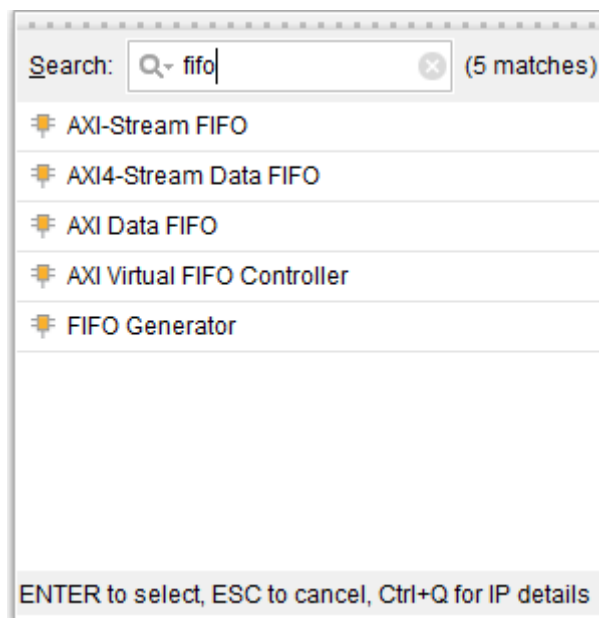
3. Click **OK**.

The IP Integrator workspace opens and, in the information area, invites you to begin adding IP.

4. Right-click in the IP Integrator design canvas and select Add IP.

The IP Integrator IP Catalog opens, displaying a list of IP available in the IP Integrator.

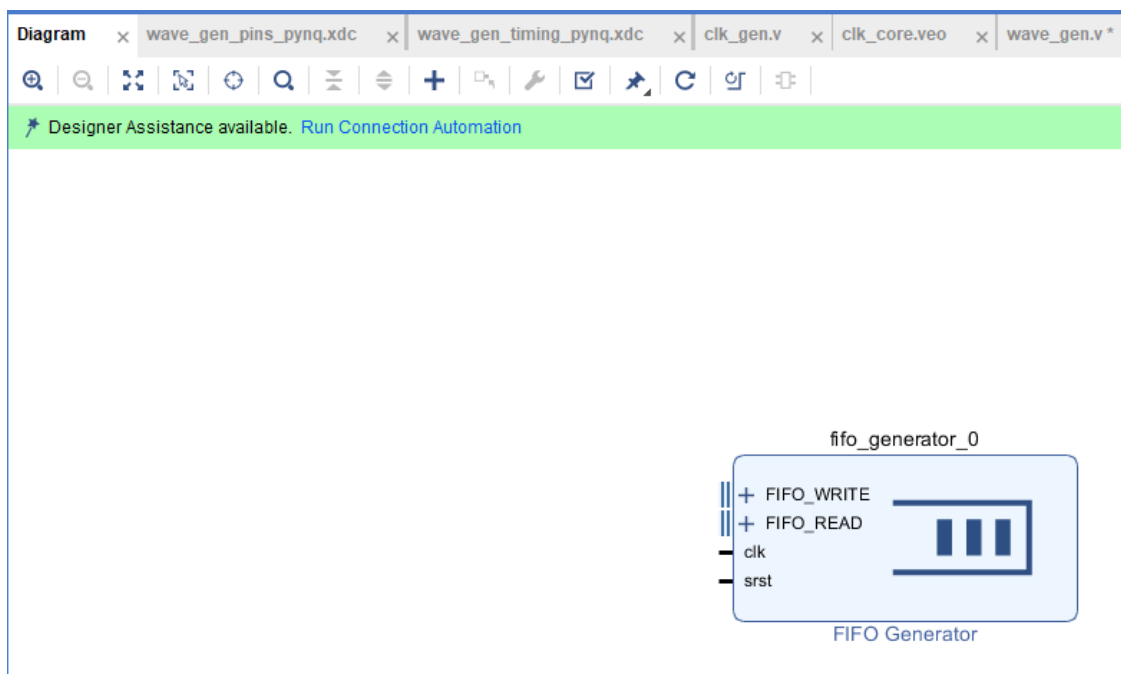
5. Type FIFO in the search box at the top of the IP Integrator Catalog to see FIFO related available IPs.



Searching for an IP in the IP Catalog

6. Double-click FIFO Generator.

The FIFO is added to the IP Integrator design canvas.



FIFO Generator instantiated

Customize the FIFO Generator IP instance.

1. Double-click the FIFO Generator IP.

The FIFO Generator displays in the Re-customize IP dialog box.

2. Make sure that the default Native option is selected for the interface type.
3. Select Independent Clocks Block RAM from the Fifo Implementation drop-down list.

Re-customize IP

FIFO Generator (13.2)

Documentation IP Location

☐ Show disabled ports

Component Name: `fifo_generator_0`

Basic Native Ports Status Flags Data Counts Summary

Interface Type

☒ Native ☐ AXI Memory Mapped ☐ AXI Stream

Fifo Implementation: **Independent Clocks Block RAM**

Synchronization Stages: **2**

FIFO Implementation Options

Supported Features

	Memory Type	(1)	(2)	(3)	(4)	(5)
Common Clock (CLK)	Block RAM	✓	✓		✓	✓
Common Clock (CLK)	Distributed RAM		✓			
Common Clock (CLK)	Shift Register					
Common Clock (CLK)	Built-in FIFO		✓	✓	✓	✓
Independent Clocks (RD_CLK, WR_CLK)	Block RAM	✓	✓		✓	✓
Independent Clocks (RD_CLK, WR_CLK)	Distributed RAM		✓			
Independent Clocks (RD_CLK, WR_CLK)	Built-in FIFO		✓	✓	✓	✓

(1) Non-symmetric aspect ratios (different read and write data widths)
 (2) First-Word Fall-Through
 (3) Uses Built-in FIFO primitives
 (4) ECC support
 (5) Dynamic Error Injection

OK Cancel

Configuring BRAM for separate read and write clocks

4. Select the Native Ports tab.

From the *Native Ports* tab you can configure the read mode, built-in FIFO options, data port parameters, and implementation options.

5. Select First Word Fall Through as the read mode.
6. Set the write width to be 8 bits.
7. Click in the *Read Width* field to change it automatically to match the write width.
8. Deselect the *Enable Safety Circuit* option.

The screenshot shows the 'FIFO Generator (13.2)' configuration window. On the left, a block diagram shows the component with ports: `FIFO_WRITE`, `FIFO_READ`, `rst`, `wr_clk`, and `rd_clk`. The main configuration area is divided into tabs: Basic, Native Ports (selected), Status Flags, Data Counts, and Summary. The 'Native Ports' tab contains the following settings:

- Component Name:** `fifo_generator_0`
- Read Mode:** ☐ Standard FIFO, ☒ First Word Fall Through
- Data Port Parameters:**
 - Write Width:** 8 (range 1,2,3...1024)
 - Write Depth:** 1024 (Actual Write Depth: 1025)
 - Read Width:** 8 (range 1,2,3...1024)
 - Read Depth:** 1024 (Actual Read Depth: 1025)
- ECC, Output Register and Power Gating Options:**
 - ☐ ECC (Hard ECC selected)
 - ☐ Single Bit Error Injection
 - ☐ Double Bit Error Injection
 - ☐ ECC Pipeline Reg
 - ☐ Dynamic Power Gating
 - ☐ Output Registers (Embedded Registers selected)
- Initialization:**
 - ☒ Reset Pin
 - ☒ Enable Reset Synchronization
 - ☐ Enable Safety Circuit
 - Reset Type:** Asynchronous Reset
 - Full Flags Reset Value:** 1
 - ☒ Dout Reset Value: 0 (Hex)
- Read Latency:** 0

At the bottom right are 'OK' and 'Cancel' buttons.

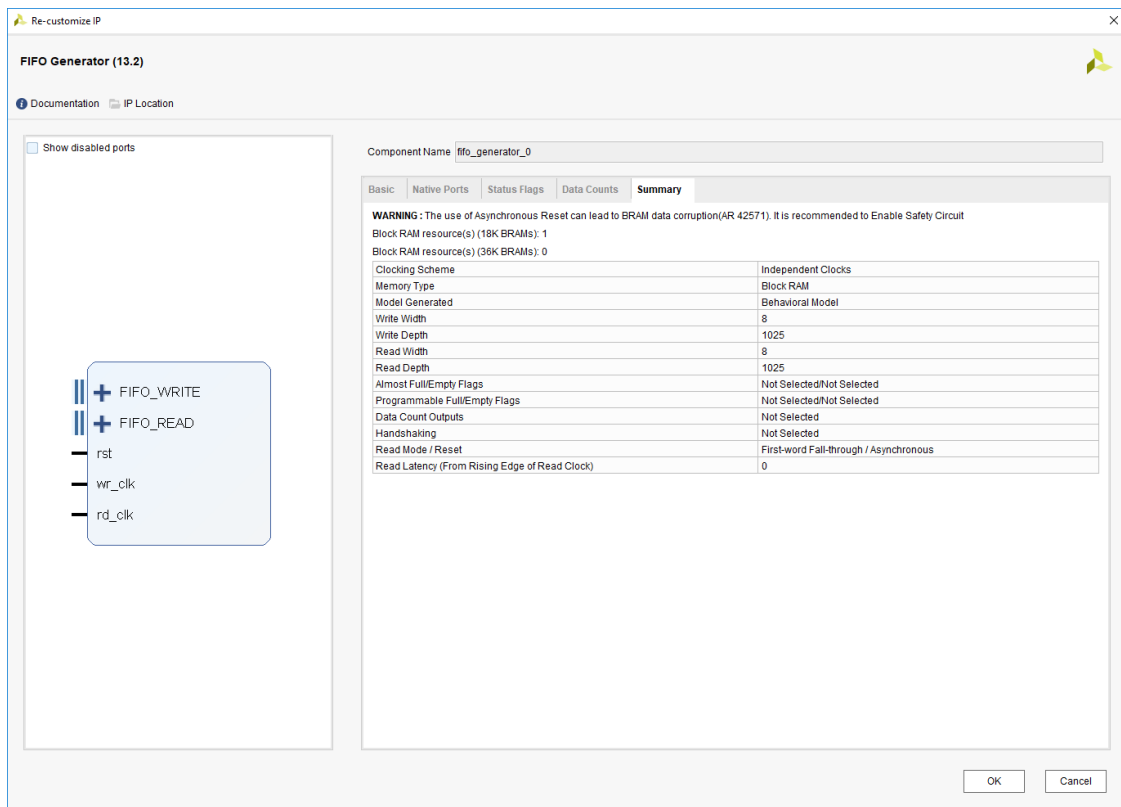
Configuring port width and read mode

9. Browse through the settings of the Status Flags and Data Counts tabs.

These tabs configure other options for the FIFO Generator. For this design, leave everything at their default settings

10. Select the Summary tab.

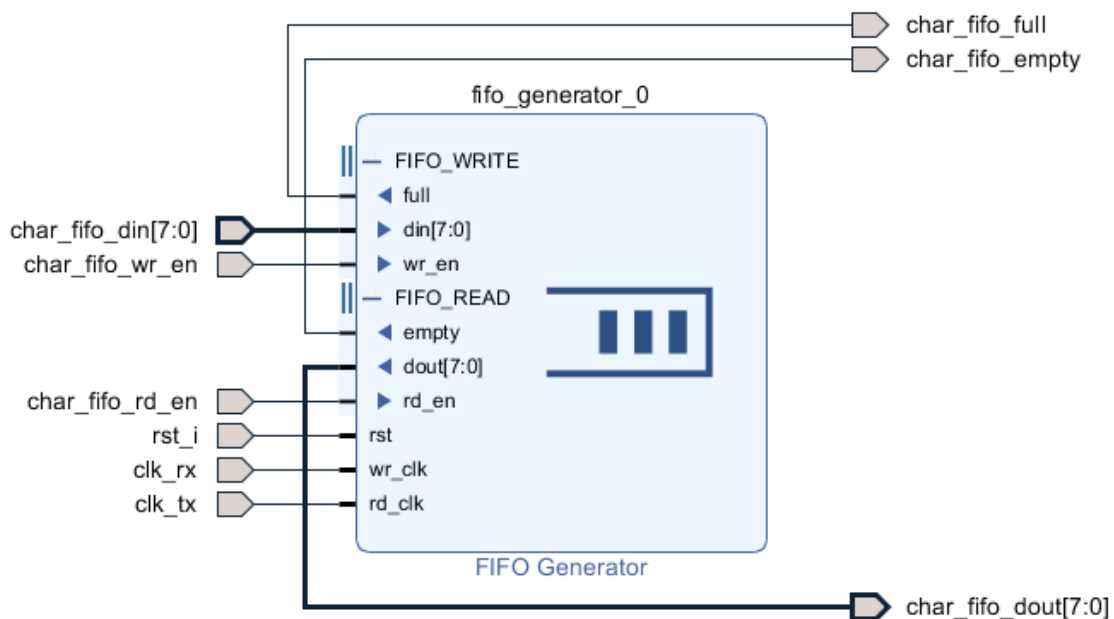
This tab displays a summary of all the selected configuration options, as well as listing resources used for this configuration.



Summary page

- Verify that the information is correct. For this configuration you are using one 18K block RAM. Click OK.

Make the ports external naming them as shown below.

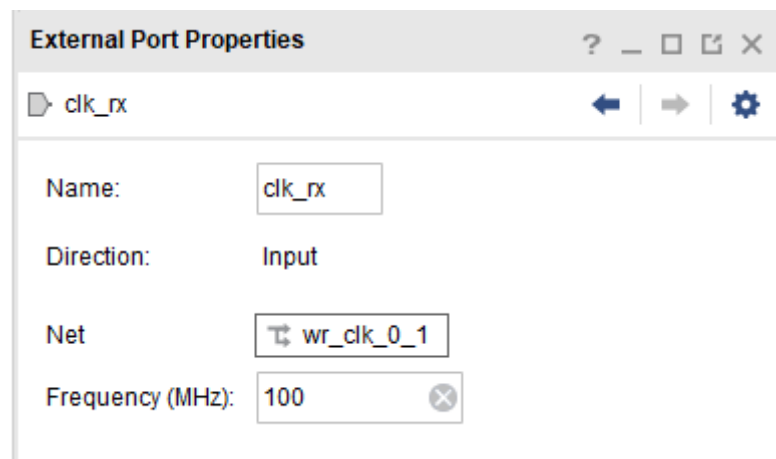


FIFO Generator IP fully generated and connected

1. Expand the **FIFO_WRITE** and **FIFO_READ** interfaces.
2. Select *wr_clk* and then press and hold the Ctrl key and select the *rd_clk* ports of the FIFO.
3. With the ports highlighted, right-click and select **Make External**.

Two external connections are created for the selected FIFO ports. Notice that the external connections have the same name as the IP module port that they connect to. You can rename these connections by selecting them and changing the name in the External Port Properties window.

4. Select the external connection port named **wr_clk**.
5. In the *External Port Properties* window, in the Name field of the General tab, type the name **clk_rx** and press Enter. Similarly, select the external connection port named **rd_clk** and change its name to **clk_tx**.



Changing the external port name

6. You will need to expand *FIFO_WRITE* and *FIFO_READ* to see the signal names by clicking on the "+" symbols next to the bus names. While pressing the Ctrl key, click all of the remaining FIFO input and output ports and make them external.
7. Change their names as listed below:

din = char_fifo_din

dout = char_fifo_dout

empty = char_fifo_empty

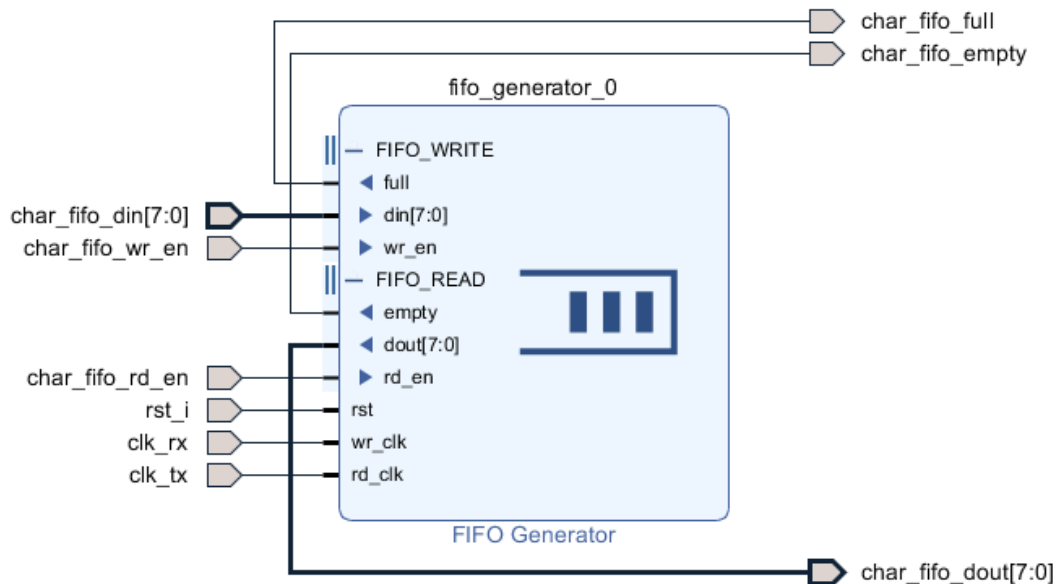
full = char_fifo_full

rd_en = char_fifo_rd_en

wr_en = char_fifo_wr_en

rst = rst_i

When you have finished, your subsystem design should look like the figure below.



Renamed external ports

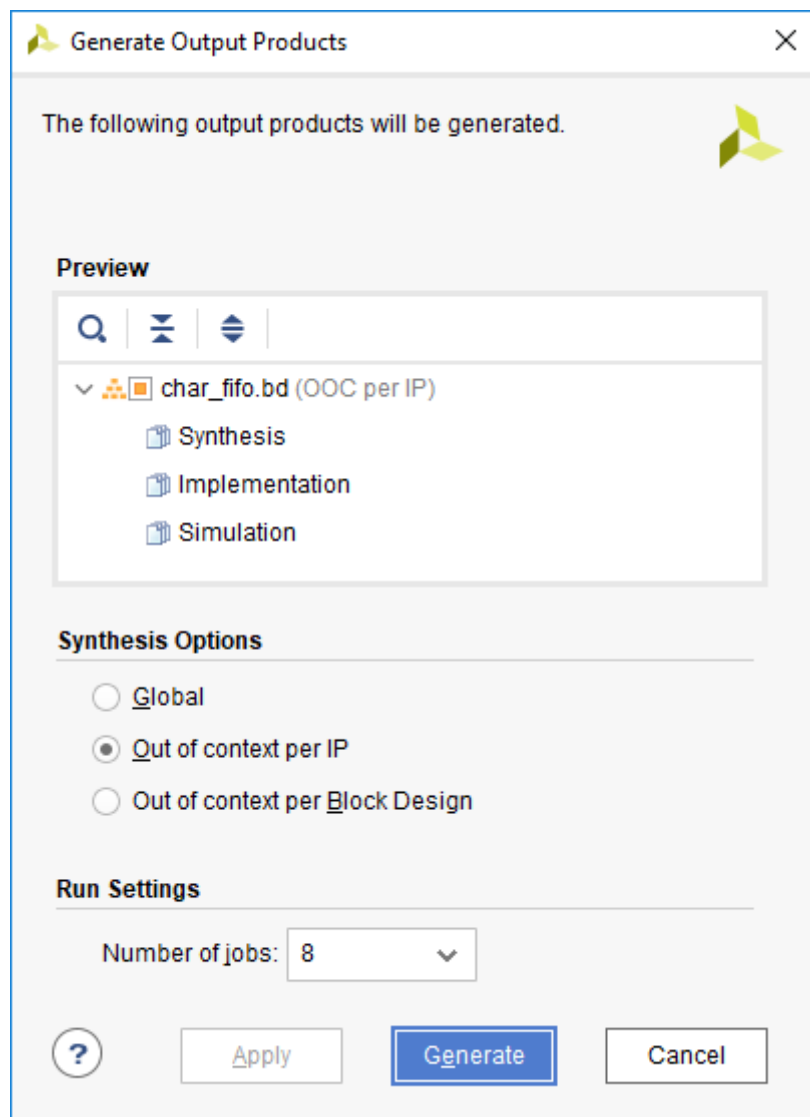
8. Click on Regenerate Layout () icon from the vertical toolbar to see the above diagram.

9. Select **Tools > Validate Design**.

You should see a message that validation was successful.

Generate the output product.

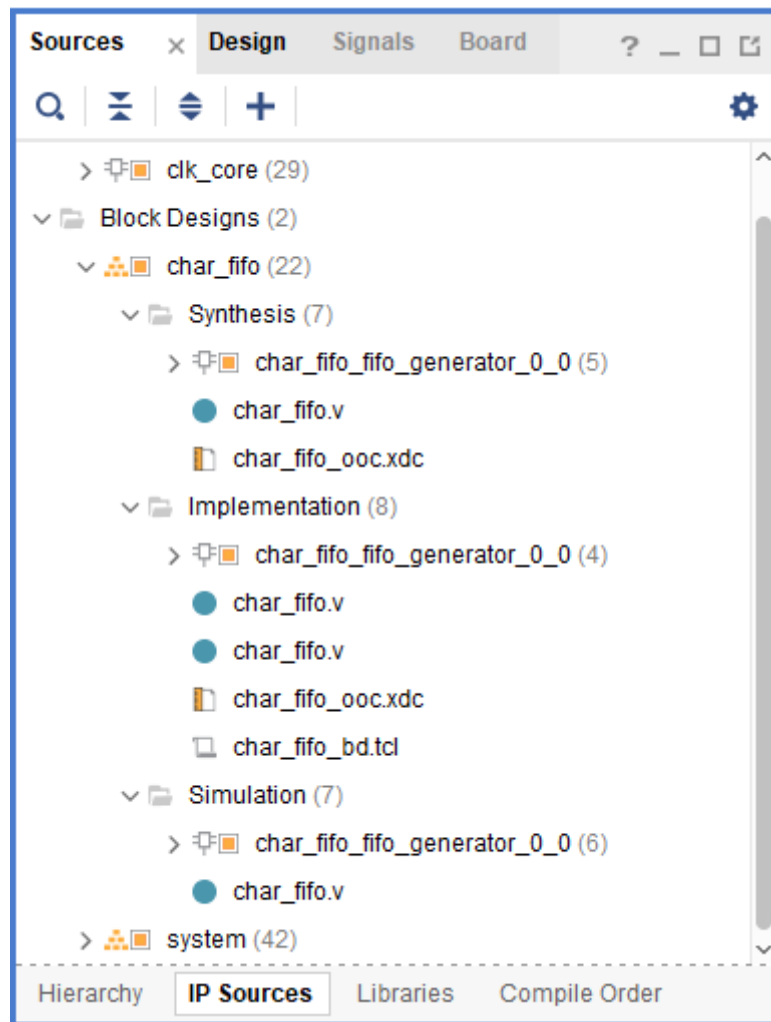
1. In the IP Sources tab of the Sources window, select the char_fifo under the Block Designs.
2. Right-click and select Generate Output Products.



Generating the output products so the IP can be instantiated in the design

3. Click Generate and **OK** to generate the output products.

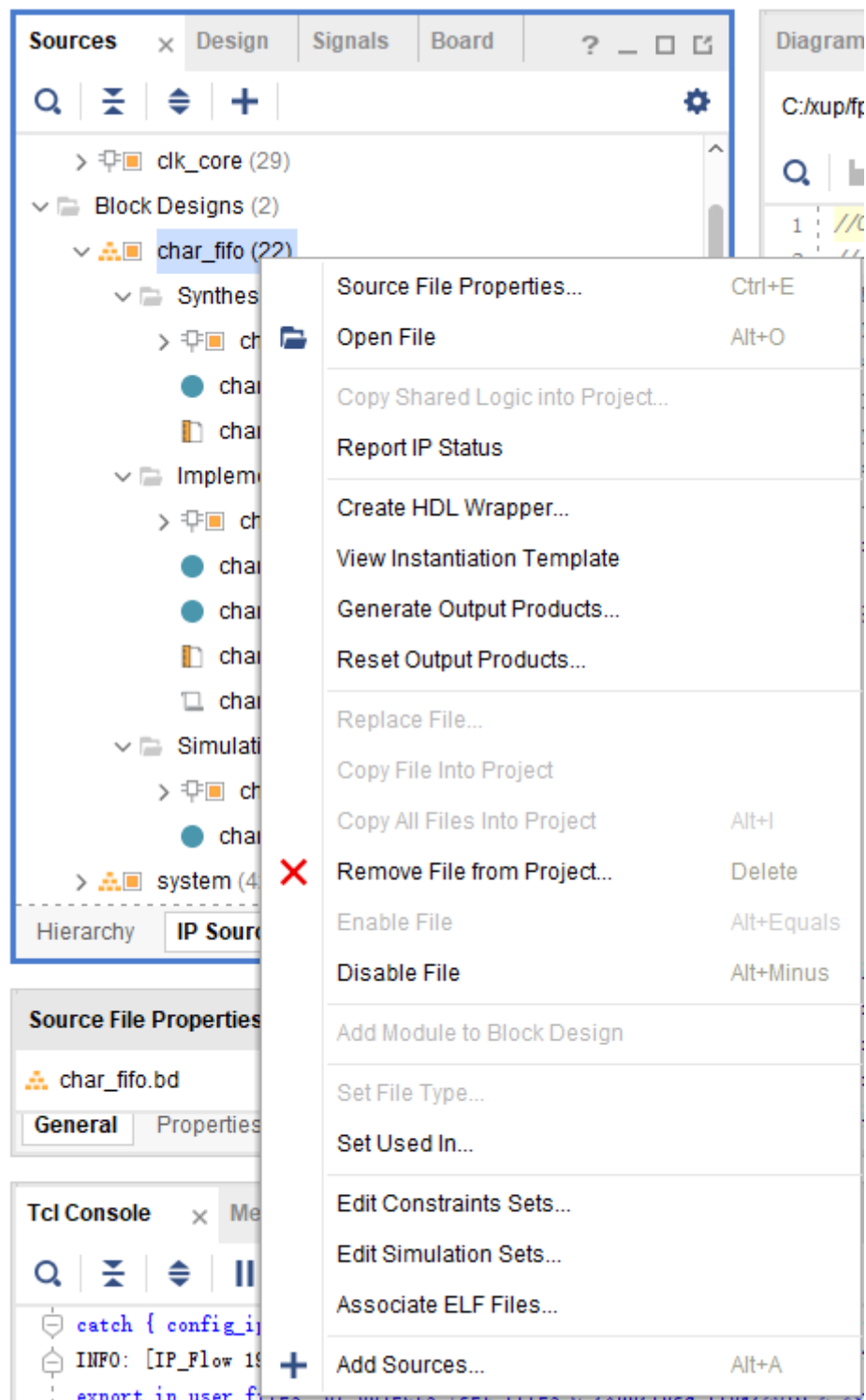
You should see the various IP output products displayed in the IP Sources tab of the Sources window.



Generated output products

Instantiate the char_fifo IP in the project.

1. From the *IP Sources* tab of the *Sources* window, select the *char_fifo* module.
2. Right-click and select *View Instantiation Template*.



Generating an instantiation template

The char_fifo_wrapper.v instantiation template is opened in the text editor in the Vivado IDE.

```

42 | char_fifo char_fifo_i
43 |     (. char_fifo_din(char_fifo_din),
44 |      . char_fifo_dout(char_fifo_dout),
45 |      . char_fifo_empty(char_fifo_empty),
46 |      . char_fifo_full(char_fifo_full),
47 |      . char_fifo_rd_en(char_fifo_rd_en),
48 |      . char_fifo_wr_en(char_fifo_wr_en),
49 |      . clk_rx(clk_rx),
50 |      . clk_tx(clk_tx),
51 |      . rst_i(rst_i));

```

Part of the instantiation template

3. Copy lines 42 through line 51, and paste them at or around line 334 in the wave_gen.v file.
4. Save the Verilog file.

Generate the bitstream and verify the functionality in hardware.

1. Click on the **Run Implementation** in the *Flow Navigator* pane.
If prompted, click **Yes** to Save the project before proceeding.
2. Click **OK** to re-run the synthesis process, followed by clicking **Save** to save the design.
3. When completed, generate the resource utilization report and verify that one FIFO is being used.
4. In the Flow Navigator, under Program and Debug, click Generate Bitstream.
5. Open the hardware manager and program the FPGA.
6. Open **SDK** by selecting **Start > Xilinx Design Tools > Xilinx SDK 2018.2**
7. In the **Select a workspace** window, click on the browse button, browse to `C:\xup\fpga_flow\2018_2_zynq_sources\lab4\pynq\lab4.sdk` and click **OK**.
8. Click **OK**.
In the *Project Explorer*, right-click on the wave_gen_uart, select *Run As*, and then **Launch on Hardware (System Debugger)**
verify the functionality of the design in the hardware.
9. When done, close the **Vivado** program by selecting **File > Exit** and click **OK**.
10. Close the **SDK** program by selecting **File > Exit** and click **OK**.

Conclusion

In this lab, you learned how to add an existing IP during the project creation. You also learned how to use IP Catalog and generate a core. You then instantiated the core in the design, implemented the design, and verified the design in hardware. You also used the IP Integrator capability of the tool to generate a FIFO and then use it in the HDL design.